

Testing Graph Databases with Synthesized Queries

Anonymous Author(s)

Abstract

Graph databases (GDBs) are increasingly used in many applications. However, their advanced features make them prone to logic bugs. Despite recent advances in GDB testing, a common limitation of existing approaches is the lack of ground truth for their test oracles. This results in both incorrectly identified bugs and overlooked bugs.

We present GQS (Graph Query Synthesis), the first automatic testing approach for detecting logic bugs in GDBs based on an established ground truth. GQS works by synthesizing a query from a randomly selected set of properties of the given graph, which serves as the ground truth. When the query is processed by the GDB under test on the full graph, any discrepancy between the actual result set and the selected property set signals a logic bug. To extensively test a GDB, we develop novel techniques that synthesize both syntactically and semantically complex queries. We implement GQS in a tool that incorporates the first Cypher query synthesizer specifically designed for testing GDBs. Overall, we find 36 previously unknown bugs across four production GDBs, of which 26 are logic bugs, with some remaining undetected for up to five years. Additionally, our tool demonstrates superior effectiveness in bug detection compared to the state-of-the-art testers.

CCS Concepts

• Information systems → Data management systems; • Software and its engineering → Software testing and debugging.

Keywords

Database Testing, Graph Database, Logic Bug

1 Introduction

Graph databases (GDBs) are increasingly used across a variety of modern applications. Unlike traditional relational databases, GDBs are optimized for managing the storage and retrieval of graph-structured data with nodes (e.g., users), relations (e.g., friendships), and properties attached to them (e.g., names for users and “since when” for friendships). This makes GDBs particularly well suited for applications such as recommender systems, social networks, and data mining [7], where intricate relationships and patterns in large datasets must be rapidly discovered and analyzed.

Unfortunately, GDBs’ advanced features make them prone to *logic bugs*, which result in them *silently* producing incorrect query results. Figure 1 presents an example of a bug discovered in FalkorDB that had existed for four years. FalkorDB returns a result that is incorrect as it has a wrong value in the `a2` field. Such logic bugs are widely recognized as being more challenging to detect than database crashes or exceptions [19, 44, 51, 57].

State of the Art. A key challenge in automatically detecting logic bugs is to develop an effective *test oracle* for determining whether a GDB correctly answers a given query. Recent research has produced excellent testers [17, 19, 22, 29, 56, 57] that attempt to address this challenge. They all test databases using randomly generated queries

and rely on either a differential [30] or a metamorphic [10] test oracle to determine if a query result is correct. A common limitation of these efforts is the *absence of ground truth* for their test oracles, resulting in both false positives (incorrectly identified bugs) and false negatives (overlooked bugs).

Differential testers [17, 56] execute the same query on different GDBs or across different versions of the same GDB. Individual execution results serve as an oracle for each other, and any discrepancy suggests a bug. Consequently, bugs that arise from libraries shared by multiple database implementations under test may be missed, e.g., two query results are the same but both are incorrect. In addition, false positives may also occur when queries yield inconsistent yet expected results for different GDBs (due to their intended designs or implementations [31, 34]). For instance, the Grand tool [56] was recently found to report a large number of false alarms [22, 57].

Metamorphic testers [19, 22, 29, 57] avoid false positives by design. They rewrite the originally generated query into a set of correlated new queries and then check if their outputs maintain the same correlation, e.g., whether one result set is equivalent to another [29], is a subset of another [19], or is a union of multiple other result sets [22, 57]. However, many bugs can still go unnoticed because these testers focus solely on relationships between the answers to different queries, rather than establishing a ground truth for each individual query. For example, suppose the query Q' is derived from the query Q based on the subset relation, and their query result sets are R' and R , respectively. While $R' \subseteq R$ passes the test, both result sets could be incorrect.

Additionally, these metamorphic testers rely on oracles that may constrain the complexity of test queries, which is desirable to trigger subtle logic bugs. For instance, GDBMeter [22] bases its test oracle on three-valued logic, which can be used only to filter clauses like `WHERE` (in Cypher language [12]). Consequently, bugs not rooted in incorrect predicate handling are likely to be overlooked.

Our Approach. We present GQS (Graph Query Synthesis), the first automatic testing approach for detecting logic bugs in GDBs based on an established ground truth. The key idea is, given the graph, to synthesize a *complex* graph query from a set of extracted properties (and their values) of randomly picked nodes and relations in the graph. This set of properties, called a *property set*, serves as the ground truth. When the query is processed by the GDB under test over the full graph, any discrepancy between the actual result set and the property set signals a logic bug.

To effectively test GDBs and uncover their logic bugs, our GQS approach aims to synthesize test queries that are both syntactically and semantically complex. These queries challenge GDBs beyond simple use cases, increasing the likelihood of revealing hidden bugs related to query parsing, optimization, graph traversal, etc. We generate these queries across four dimensions: (i) incorporating diverse search patterns, (ii) generating deeply nested expressions within a query, (iii) establishing highly correlated data dependencies between different parts of the query, and (iv) utilizing a wide range of language features, including various clauses and functions.

```

1 MATCH (n0 :L1 :L2) <-[r0]-(n1) <-[r1]-(n2 :L1) <-[r2]-(n3)
2 WHERE (((((NOT (n2.k16 CONTAINS n0.k14)) ...
3 UNWIND [n1.k6 <-> r2.id AND n2.k14 ENDS WITH n2.k15] AS a0
4 OPTIONAL MATCH (n0) <-[r5]-(n1) <-[r6]-(n2) <-[r7]-(n3) ...
5 WITH n0, r5, r7.k74 AS a2, count(replace(.....)) AS a4,
6 RETURN n0.id AS a0, r5.id AS a2
7 // expected result: {a0:4, a2:10} ✓
8 // actual result: {a0:4, a2:13} ✗

```

Figure 1: A logic bug found in FalkorDB. The bug-triggering query in Cypher involves various search patterns and nested expressions (underlined), as well as different types of clauses and cross-clause variable references (colored). Parts of this query are omitted for simplicity.

The query shown in Figure 1 is synthesized by GQS, which uncovered a previously unknown logic bug in FalkorDB. This test query incorporates various search patterns and deeply nested expressions, as underlined. Additionally, it utilizes a variety of clause types, such as `MATCH`, `UNWIND`, and `WITH`, as well as functions like `count`, and exhibits complex cross-clause dependencies (e.g., the variable `n0` is referenced across four different clauses), as highlighted.

Synthesizing complex GDB queries like that in Figure 1 from a property set is, however, highly non-trivial. Over the past decade, there have been numerous advances in database query synthesis [32, 33, 39, 46, 49, 52, 53, 55]. Yet, these synthesizers are *not* suitable for testing GDBs because they all generate SQL or datalog queries, which do not have a graph semantics. Moreover, adapting their methodologies is challenging because these synthesizers are not designed to extensively test databases. For example, simple queries are preferable in many cases [53, 55].

To effectively trigger logic bugs in GDBs, GQS synthesizes complex queries in a *stepwise* manner, using a collection of novel techniques. Specifically, we leverage the *chaining* structure of graph queries to devise a synthesis plan for the given property set, breaking the synthesis task into individual steps, each corresponding to a specific clause in the final query. In particular, these steps involve synthesizing queries that go beyond the simple `MATCH-RETURN` structure, allowing for the integration of a wider range of clauses and functions. For instance, as shown in Figure 1, although the desired result includes only `n0.id` and `r5.id` (line 6), additional synthesis steps are involved, such as introducing more nodes and relations in the `MATCH` step (line 1), distributing lists in the `UNWIND` step (line 3), and projecting in the `WITH` step (line 5). This also facilitates creating rich data dependencies across clauses, e.g., the referenced variables in the same colors. To ensure the final query result remains consistent with the ground truth, GQS schedules these additional synthesis steps in a *pairwise* manner, such as by removing any extra nodes or properties introduced earlier.

Additionally, we devise two techniques to enhance the complexity of the synthesized clause at each individual step. First, through *pattern mutation*, we instantiate the clause with varying search patterns, such as the two `MATCH` steps (lines 1 and 4). Second, to examine a wide range of expression features, including various functions and operators, we construct *branching and nested* expressions embedded within the clause, e.g., the `WHERE` predicate and the `count` function (lines 2 and 5).

We implement our GQS approach in a tool that incorporates the first Cypher query synthesizer tailored for extensively testing

GDBs for logic bugs. We chose Cypher as it is the most widely adopted, fully-specified graph query language [35], and it is used by prominent GDBs like Neo4j [34] and Memgraph [31]. However, our approach is general and can be adapted to test GDBs using other query languages, as discussed in Section 7.

Contributions. Overall, we make the following contributions.

- At the conceptual level, we provide a novel approach to tackle the test-oracle problem of detecting logic bugs in GDBs by synthesizing test queries based on an established ground truth.
- At the technical level, we propose a stepwise synthesis approach called GQS, incorporating novel techniques that ensure the synthesized test queries are complex, thereby stressing GDBs.
- At the practical level, we realize GQS in an automatic testing tool and assess it on four extensively tested production GDBs supporting Cypher queries: Neo4j, Memgraph, Kùzu [24], and FalkorDB [13]. Our tool discovers 36 new bugs, including 26 logic bugs, with some remaining latent for up to five years. Compared to the state-of-the-art testers, our tool also exhibits superior effectiveness in bug detection.

We have open-sourced our approach and synthesizer to support further research on GDB testing and related applications.

2 Background

2.1 GDBs and Labeled Property Graphs

Graph databases (GDBs) structure data as a graph $G = \langle N, R \rangle$, where N denotes nodes (or objects) and R denotes relations (or relationships). The nodes and relations are also associated with labels or types, which define their categories. Furthermore, additional information may be associated with them via properties. Following the Cypher Reference [37], we define a property p as a key-value pair (k, v) . The key k is a tuple $\langle e, n \rangle$, where $e \in (N \cup R)$ specifies the graph element, and $n \in P_n$ is the property name. Accordingly, $v \in V$ is the value. The sets P_n and V are domain dependent. This structure, known as a *labeled property graph* (LPG) [9], is widely used in GDBs such as Neo4j and Memgraph.

Example 2.1. Figure 2 illustrates an example of an LPG modeling users' movie preferences. Specifically, the node N_1 labeled `USER` is connected to two nodes, N_2 and N_3 , labeled `MOVIE`, via relationships R_1 and R_2 , both labeled `LIKE`. The nodes and relationships include properties such as $\langle N_3, \text{genre} \rangle$ with the value `["Drama", "Romance"]` and $\langle R_2, \text{rating} \rangle$ with the value `10`.

Additionally, a *property set*, denoted as $P = \{p_1, p_2, \dots, p_n\}$, is a collection of properties attached to graph elements. This serves as the ground truth for validating the query results. For instance, the two properties of the above example form a property set.

2.2 The Cypher Query Language

Cypher [35] is the *de facto* query language for GDBs. A Cypher query is structured as a sequence of *clauses*. Each clause takes as input a table of intermediate status (an empty table if it is the first clause), and produces a new table of intermediate status, serving as the input to the next clause. The last clause's output is the query

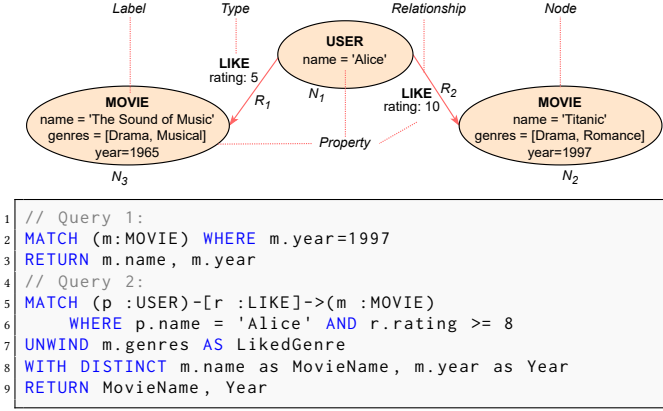


Figure 2: An example labeled property graph storing the movies that users like (top) and two examples of Cypher queries retrieving the movie’s name and year (bottom).

result. Each clause can also refer to the LPG when required, for example introducing graph elements to the table.

Cypher employs five main clauses to retrieve data, namely **MATCH**, **OPTIONAL MATCH**, **UNWIND**, **WITH**, and **RETURN**. The **MATCH** clause searches for the user-specified patterns in an LPG and introduces the matching subgraphs into the intermediate result. **OPTIONAL MATCH** allows partial matching and fills in the missing parts of the pattern with *null* values. The **UNWIND** clause expands a list into individual rows, enabling subsequent clauses to perform operations on each element within the list. The **WITH** clause allows manipulating the intermediate result before passing it to the following clauses, for example, introducing new variables through projections or removing existing variables in the intermediate table by excluding them from the projection list. The **RETURN** clause has the same semantics as **WITH** but is used as the final clause in queries.

Additionally, the **UNION** and **CALL** clauses can be combined with the above clauses. The **UNION** clause connects two queries and computes the union of their outputs; the **CALL** clause invokes database engine procedures. Subclauses like **WHERE**, **ORDER BY**, **SKIP**, and **LIMIT** function as refinement operations within clauses. For instance, the **WHERE** clause adds constraints to the patterns described in the **MATCH** clause or filters the results in the case of **WITH**; the **ORDER BY** clause sorts results; the **LIMIT** and **SKIP** clauses truncate result sets. These subclauses mirror SQL functionalities.

Example 2.2. Figure 2 shows two Cypher queries that retrieve the same property set with keys $\langle N_2, name \rangle$ and $\langle N_2, year \rangle$. The first query employs a simple **MATCH-RETURN** structure. The second query returns the same result but the query has more complex features. The **MATCH** clause (line 5), together with the **WHERE** clause (line 6), searches for the movies liked by Alice and rated at least 8. Note that variables, such as *p* and *r*, bind to the nodes and relations, such as the nodes with **USER** label and relations with **LIKE** type, respectively. The **UNWIND** clause (line 7) then expands the list of genres associated with the matching movies into individual rows (two in this case) under the column *LikedGenre*. The **WITH** clause (line 8) projects the node property like name as *MovieName* and removes duplicates with the **DISTINCT** operator. Finally, the **RETURN** clause (line 9) returns the query result.

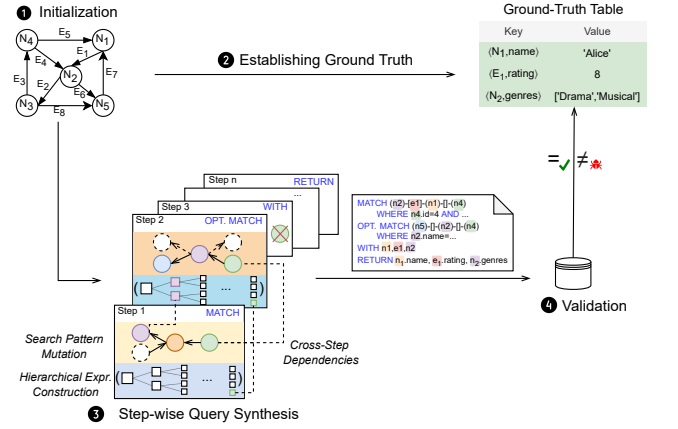


Figure 3: The workflow of GQS.

Cypher also uses six additional clauses for writing data to the database, including **CREATE** (for creating graph elements), **SET** (for updating properties), **MERGE** (which acts as a combination of **MATCH** and **CREATE**, allowing conditional actions depending on whether the specified data already exists or is newly created), **DELETE**, **DETACH DELETE** and **REMOVE** (for removing graph elements or properties). Unlike data retrieval clauses, which are utilized by test oracles to determine if the GDB under test correctly answers a given query, these clauses are mainly used for initializing or updating graphs.

Further details about all these clauses can be found in the Cypher Query Language Reference [37].

3 Graph Query Synthesis

In this section, we present GQS (Graph Query Synthesis), an automatic testing approach for detecting logic bugs in GDBs. GQS is unique in its ability to establish ground truth for complex graph queries that are used to test these databases extensively.

3.1 Overview

Figure 3 illustrates GQS’s workflow, consisting of four main steps.

1 Initialization. We begin by randomly generating a graph. For instance, the graph in Figure 3 comprises five nodes and eight relations. Additionally, labels and properties are randomly assigned to the nodes and relations, and indexes are created for these labels and properties. The graph is then loaded into the GDB under test.

2 Establishing Ground Truth. We randomly select properties from graph elements to form a *property set*, such as $\langle N_1, name \rangle$, $\langle E_1, rating \rangle$, $\langle N_2, genres \rangle$ and their corresponding values, which are then stored in a table as key-value pairs. This serves as the ground truth. Subsequently, we synthesize a graph query that is expected to produce these key-value pairs as the result upon execution.

3 Step-wise Query Synthesis. We devise a *synthesis plan* consisting of multiple steps to obtain the property set. Each step focuses on synthesizing a part of the final query, such as the **MATCH** clause in Step 1 and the **RETURN** clause in the last step, as shown in Figure 3. To enhance the final query’s complexity, these steps combine a variety of clauses, such as **WITH**, extending beyond the simple **MATCH-RETURN** structure. Across different steps, data dependencies are established via variable references, as illustrated by the colors

and dashed lines in the figure, leading to semantic connections between clauses.

These clauses also introduce nodes, properties, and aliases outside the property set. To guarantee that the final query result still aligns with the property set, we schedule these clauses in a *pairwise* manner. For instance, the extra blue node added at Step 2 (i.e., n_5 in the query) using **OPTIONAL MATCH** is removed at Step 3 using **WITH** (i.e., excluded from the projection list).

At each step, we increase the complexity of the generated clause by mutating its search patterns to explore different query plans in the GDB engine, such as graph traversal processes, as illustrated by the dashed arrows and nodes in Steps 1 and 2. Furthermore, we embed hierarchical expressions within the generated clause, incorporating branching and nested structures, to examine various GDB functions, operators, etc. Finally, we chain all the clauses together to form the complete query.

4 Validation. As the final step, we execute the synthesized query against the GDB under test. Any discrepancy between the actual execution result and the ground truth established at Step 2 indicates the presence of a logic bug.

The above four steps illustrate one iteration of our testing process, which can be repeated. In subsequent iterations, we can either proceed with Step 3 to synthesize another graph query, potentially with a different number of steps; continue with Step 2 to generate new queries for another extracted properties; or restart with Step 1 to create a new GDB, which may vary in size. These actions are chosen randomly. Before testing begins, we configure both the number of iterations and the number of synthesis steps per iteration.

At the core of our GQS approach is the synthesis of complex queries from a given property set (Step 3). The remainder of this section provides a detailed explanation of this process.

3.2 Integrating Diverse Clauses

To synthesize a query that obtains the established ground truth, i.e., the selected property set, a simple **MATCH-RETURN** structure is sufficient, as shown in the first query of Figure 2. However, to extensively test the GDB, more complex queries are desirable. Therefore, GQS incorporates additional clauses during query synthesis, distributing them across steps. These additional clauses introduce diverse graph elements and aliases beyond the ground-truth property set, to build more cross-step dependencies, enable different search patterns, etc. To ensure the final query still retrieves the desired property set, GQS schedules these clauses in a *pairwise* manner, e.g., by removing the extra nodes or properties added earlier.

Based on the semantics of the five main clauses related to data retrieval [37], we devise the following three pairs of “add and subtract” operations, as also illustrated in Table 1.

- **Introducing/Removing Elements.** Graph elements, such as nodes and properties, are introduced using **(OPTIONAL) MATCH** clauses. They can be removed using the **WITH** or the **RETURN** clause by excluding them from the intermediate or final result set, respectively.
- **Creating/Removing Aliases.** Aliases can be bound to expressions, including those for property accesses (e.g., $n_0.id$ is bound to the alias a_0 as shown in Figure 1), and to existing elements, serving as referenceable variables in the query.

Table 1: Paired operations for adding and removing elements and alias, as well as expanding and truncating lists.

Notation	Operation	Clause
$\mathcal{E}^+$	introducing elements	(OPTIONAL) MATCH
$\mathcal{E}^-$	removing elements	WITH, RETURN
$\mathcal{A}^+$	creating alias	WITH, RETURN
$\mathcal{A}^-$	removing alias	WITH, RETURN
$\mathcal{L}^+$	expanding lists	UNWIND
$\mathcal{L}^-$	truncating lists	WITH, RETURN

Both **WITH** and **RETURN** clauses can create or remove these aliases.

- **Expanding/Truncating Lists.** A list of graph elements can be expanded into separate rows using the **UNWIND** clause. This duplicates the table of intermediate status based on the list’s length, assigning each list element to one of the copies with a new alias. To truncate the expanded list and retain a single copy, the **WITH** or **RETURN** clause along with result refinement subclauses like **LIMIT** and **WHERE** can be applied.

To arrange these paired operations across individual steps, we classify them into two categories: (i) essential operations, which introduce properties in the property set, and (ii) supplementary operations, which are not directly tied to the property set.

Specifically, for each property key $\langle \mathcal{E}, p \rangle$ in the property set, there are two types of essential operations. The first type involves the introduction of the graph element itself, denoted as $\mathcal{E}^+$, since accessing the property relies on the existence of the element. The second type refers to the actual access to the property, denoted as $(\mathcal{E}.p)^+$. Since the element $\mathcal{E}$ itself is not part of the ground truth, a paired removal, denoted as $\mathcal{E}^-$, is also scheduled.

Supplementary operations involve elements unrelated to the ground truth. This includes introducing extra graph elements (e.g., nodes, relations, etc.), creating aliases (bound to these graph elements or random expressions), and expanding randomly constructed lists. As with Category (i), each of these operations is paired with a corresponding removal operation.

Example 3.1. Consider the initial graph in Figure 3 and the designated ground truth with property keys $\langle N_1, name \rangle$, $\langle E_1, rating \rangle$, and $\langle N_2, genres \rangle$. Category (i) involves nine essential operations, i.e., N_1^+ , $(N_1.name)^+$, N_1^- , E_1^+ , $(E_1.rating)^+$, E_1^- , N_2^+ , $(N_2.genres)^+$, and N_2^- , where three pairs of add-subtract operations are arranged for N_1 , E_1 , and N_2 that are not part of the property set.

Operations in Category (ii) are then supplemented by GQS. For example, an array l associated with the expression $[E_4.rating]$ can be expanded; an alias a bound to a property-access expression $N_2.name$ can also be introduced. The involved operations include l^+ , E_4^+ , and a^+ and their paired removals. Note that N_2^+ and N_2^- have already been arranged by the essential operations.

3.3 Building Cross-Step Dependencies

Our GQS approach distributes all operations across steps to build complex cross-step dependencies through referenceable variables in search patterns or expressions. However, *valid* dependencies are established only when operations are scheduled in the correct order. For instance, the property-access operation $(N_1.name)^+$ can

only be assigned to a step after N_1 has been introduced in an earlier step; otherwise, the dependency cannot be formed as the query is syntactically invalid.

When scheduling operations, two key temporal constraints must be considered to build valid dependencies. First, in any task, the graph elements involved must have been introduced before they are referenced. Second, removal operations must be scheduled after their corresponding add operations.

We denote these temporal constraints as $O \prec O'$, meaning that the operation O must be scheduled before the operation O' . In the above example, the constraint $N_1^+ \prec (N_1.name)^+$ must be satisfied. Additionally, as Cypher allows element removal to be scheduled in the same clause as alias binding [37], we also account for weak constraints, denoted as $\preceq$. For example, $(N_1.name)^+ \preceq N_1^-$ means that the removal of the element N_1 can be scheduled either at the same step or after its property access $N_1.name$.

Example 3.2. Overall, the seven constraints for establishing valid dependencies in Example 3.1 are:

- | | |
|--|-------------------------------------|
| (1) $N_1^+ \prec (N_1.name)^+ \preceq N_1^-$ | (5) $E_4^+ \prec I^+ \preceq I^-$ |
| (2) $E_1^+ \prec (E_1.rating)^+ \preceq E_1^-$ | (6) $N_2^+ \prec a^+ \preceq a^-$ |
| (3) $N_2^+ \prec (N_2.genres)^+ \preceq N_2^-$ | (7) $N_2^+ \prec a^+ \preceq N_2^-$ |
| (4) $E_4^+ \prec I^+ \preceq E_4^-$ | |

GQS schedules operations in different steps while ensuring no violation of constraints by adapting *topological sorting*. Specifically, it first scans the operations to be arranged, and then constructs a tree representing the involved constraints, where each node is an operation and each relation is the temporal order between two operations. The distribution procedure takes as input the tree and outputs, for each step, the assigned operations and the referenceable variables. These variables can then be referenced in search patterns or expressions, thus building data dependencies across steps. The pseudocode is given in Algorithm 1.

The procedure starts by traversing the tree (line 4) and collects the nodes with zero indegree. The corresponding operations are added to the candidate list for the current step (line 6), as they do not rely on any operations that have not yet been arranged. In particular, if an operation O with zero indegree is related to another operation O' by a weak constraint, i.e., $O \preceq O'$, both operations are included in the candidate list if O' has no other incoming relation (lines 7–9). Upon identifying all candidate operations for the current step, the procedure randomly selects a subset that can be realized using the same type of clause (line 14) such as **MATCH** or **WITH**, as specified in Table 1. This is because each step corresponds to a single clause. Note that for each weak constraint $O \preceq O'$, if O' is selected, then O must also be selected to avoid having O arranged at later steps than O' . Finally, these operations, along with their associated constraints, are removed from the tree (line 16).

Note that in each step, variables available for references are also updated (line 15) by aggregating variables introduced in previous steps while excluding the variables removed at this step. When synthesizing concrete clauses, these variables, including aliases, nodes, and relations can be referenced in search patterns and expressions. This further increases cross-clause dependencies.

Algorithm 1 Scheduling operations

Input: *Tree*, a tree comprising operations and their constraints
Output: *Step*, an array with the operations assigned to each step; *Var*, an array with the referenceable variables at each step

```

1: cnt ← 1                                ▷ the current step
2: while Tree is not empty do
3:   candidates ← ∅                        ▷ initializing candidate ops
4:   for op in Tree do
5:     if op.indegree == 0 then
6:       candidates.append(op)
7:       for op' in op.weak_related do      ▷ op ⪯ op'
8:         if op'.indegree == 0 then
9:           candidates.append(op')
10:        end if
11:      end for
12:    end if
13:  end for
14:  Step[cnt] ← random_subset(candidates)
15:  Var[cnt] ← ref_vars(Var[cnt − 1], Step[cnt]) ▷ Var[0] = ∅
16:  Tree.remove(Step[cnt])
17:  cnt ← cnt + 1
18: end while

```

3.4 Mutating Search Patterns

Diverse search patterns involve different ways of combining nodes and relations, as well as referencing elements introduced earlier in the query. Recall the search pattern $(p : \text{USER}) - [r : \text{LIKE}] \rightarrow (m : \text{MOVIE})$ in Figure 2 (line 5). A “reverse” yet equivalent pattern, $(m : \text{MOVIE}) \leftarrow [r : \text{LIKE}] - (p : \text{USER})$, would trigger different query plans, such as graph traversal processess, in many GDBs like Memgraph [31].¹ In the former pattern, all USER nodes are retrieved first, whereas, in the latter pattern, the traversal starts by retrieving all MOVIE nodes. Moreover, if a subsequent search pattern $(p) - [r2 : \text{DISLIKE}] \rightarrow (m2 : \text{MOVIE})$ is used, the traversal will begin specifically from the Alice node, to which the variable p is bound through the first **MATCH** clause (line 5–6), to find the movies she dislikes.

To explore GDBs’ different query-processing behaviors, GQS generates a variety of search patterns that both contain to-be-introduced graph elements and also reference already-introduced graph elements, using novel *pattern mutation* strategies.

GQS begins by collecting random patterns that include the elements to be introduced from the graph, referred to as *base patterns*. For instance, the three dashed boxes in Figure 4 depict three base patterns, each containing the nodes N_3 and N_4 to be introduced.

GQS then iterates through all these base patterns and mutates them to new patterns that reference the graph elements introduced in previous clauses. First, for each base pattern, it identifies the graph elements that also appear in the patterns used in previous clauses. For example, in the scenario ① of Figure 4, the base pattern (marked in blue) contains the to-be-introduced nodes N_3 and N_4 , along with the node N_1 that also appears in the previously used pattern (marked in yellow). There are three cases based on the position

¹Some GDBs may optimize these two patterns into the same query plan. However, this is generally not the case for complex, yet semantically equivalent, search patterns [29].

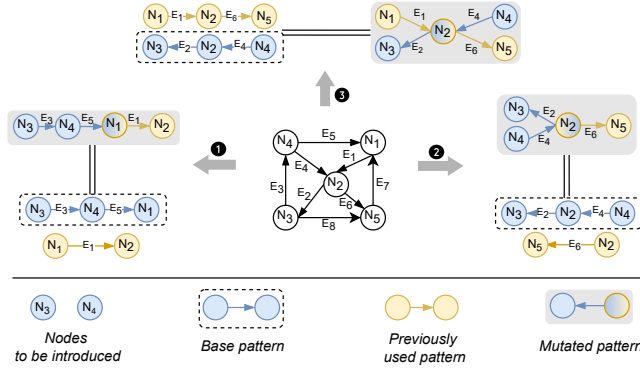


Figure 4: Three pattern mutation strategies.

of the shared element. For each, we devise a corresponding mutation strategy, which GQS applies to the base pattern, combining it with the pattern used in an earlier step.

- If the common element appears at the beginning or end of both the base and previous patterns, the two patterns are concatenated, as illustrated by ① in Figure 4. In this case, the mutated pattern now also references the previously introduced node N_2 .
- When the common element is located at the start or end of either the base or the previous pattern (but not both), a branching mutation is applied, as depicted in ②. In this scenario, the previously introduced node N_5 is incorporated into the mutated pattern, forming two branches that separately involve N_3 and N_4 .
- If the common element is located in between the two ends of both the base and previous patterns, a cross mutation applies, as shown in ③. The mutated pattern can then be split at the common element (N_2 in this case) into multiple subpatterns, which are then randomly recombined to form new search patterns.

Finally, we encode the mutated patterns into search patterns that are compatible with the **MATCH** clause. For example, the mutated pattern in the scenario ② can be encoded into two search patterns in a **MATCH** clause: $(n3) \leftarrow [e2] - (n2) - [e6] \rightarrow (n5)$ and $(n4) \leftarrow [e4] \rightarrow (n2) - [e6] \rightarrow (n5)$.

During this process, GQS introduces additional mutations to enhance the complexity of the generated queries. Specifically, the labels or types of the corresponding nodes or relations are added to the search pattern element to enable index-based optimizations by the GDBs under test. Additionally, relations may either retain their original direction or disregard direction to trigger a wider range of graph traversal processes. For instance, the first search pattern mentioned above can be mutated into $(n3:L) \leftarrow [e2] - (n2) - [e6] \rightarrow (n5)$, where L specifies the label attached to $n3$, and the relation directions are removed. Furthermore, if the search pattern is part of an **OPTIONAL MATCH** clause, which allows matched subgraphs to partially align with the pattern, it can be extended with additional random patterns. This further increases the query's complexity.

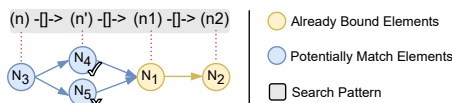


Figure 5: An illustrating example for predicate construction.

An encoded search pattern may match multiple subgraphs. For example, as shown in Figure 5, consider the pattern $(n) \leftarrow [e2] \rightarrow (n') \rightarrow (n1) \leftarrow [e4] \rightarrow (n2)$ for the scenario ①, where $n1$ and $n2$ were bound to the nodes N_1 and N_2 in previous steps. Both the subgraphs $N_3-N_4-N_1-N_2$ and $N_3-N_5-N_1-N_2$ match this pattern; however, only the former can correctly introduce the nodes N_3 and N_4 , as scheduled (i.e., N_3^+ and N_4^+). GQS filters out undesired graph elements with predicates in the **WHERE** subclauses.

To construct such predicates, first, GQS scans through the search pattern and selects a pattern element that is already bound to a graph element. In our example, $(n1)$ could be selected as it was bound to N_1 in the previous steps. If no such pattern elements exist, then one pattern element is randomly picked, and a predicate is constructed to ensure that it only matches the desired graph element. Then, starting from this selected pattern element, GQS traverses the graph following the search pattern and constructs predicates when multiple matches could occur. In our example, starting from $(n1)$, GQS first checks the pattern segment $(n') \leftarrow [e2] \rightarrow (n1)$. Both N_4 and N_5 could match (n') , as they both have an outgoing relation connected to N_1 . Therefore, a predicate, such as $n'.id = 4$, is constructed to ensure that n' is bound to N_4 (assuming identifiers are unique). This process is repeated until all the pattern segments are checked. Note that for **OPTIONAL MATCH** clauses, the randomly extended pattern segments are also checked to ensure that the pattern uniquely matches one subgraph, thereby avoiding an increase in the multiplicity of the results. Finally, the expressions used in these predicates will be further substituted with more complicated ones that also achieve the same filtering, as described in Section 3.5.

3.5 Generating Complex Expressions

While predicates with simple expressions, such as $n'.id = 4$, are sufficient to filter out undesired graph elements, more complex expressions are preferable to extensively test GDBs.

Previous work [17, 44] provides a good basis for generating complex expressions, like $\text{char_length}('abc') + \text{sqrt}(\text{round}(1.2))$, that satisfy specific value constraints, such as evaluating to 4 as in the right-hand side of the above predicate. GQS adapts the approach proposed in [17] to construct such expressions. Specifically, GQS first selects an expression template for the new expression. This template includes functions (e.g., $\text{sqrt}(\text{par})$), operator applications (e.g., $\text{par1} + \text{par2}$), and string match expressions (e.g., $\text{par1 CONTAINS par2}$), where par1 and par2 denote the parameters of the template. Such templates function as the skeleton of the expressions. Next, GQS converts the value constraint into respective sub-constraints for these parameters. For instance, if the template $\text{par1} + \text{par2}$ is selected, the sub-constraints for par1 and par2 can be evaluating to 3 and 1, respectively. This process repeats *recursively* to construct an expression for each parameter until the nesting depth is reached.

Replacing Property-Access Expressions. Compared to expressions like the above, generating complex expressions to replace property-access expressions in predicates, such as $n'.id$, is more challenging because the new expressions must still effectively filter out undesired graph elements. For example, the expression $\text{sign}(n'.id)$ cannot replace $n'.id$ as both $n'.id = 4$ (i.e., N_4)

Algorithm 2 Replacing property-access expressions

Input: S_1 and S_2 , two graph element sets; P , a common property name; D , the nesting depth

Output: Exp , an expression that, when instantiated with S_1 and S_2 , evaluates to different values with respect to P

```

1:  $Exp \leftarrow \text{original\_property\_access\_expr}(P)$ 
2:  $V_1 \leftarrow \text{property\_val}(S_1, P)$   $\triangleright$  instantiating  $Exp$  with  $S_1$ 
3:  $V_2 \leftarrow \text{property\_val}(S_2, P)$   $\triangleright$  instantiating  $Exp$  with  $S_2$ 
4:  $Type \leftarrow \text{get\_type}(Exp)$ 
5: while  $D > 0$  do
6:    $Tplt \leftarrow \text{exp\_template\_with\_param\_type}(Type)$ 
7:    $New\_Exp \leftarrow Tplt.\text{nest}(Exp)$ 
8:   if  $New\_Exp.\text{eval}(V_1) \cap New\_Exp.\text{eval}(V_2) = \emptyset$  then
9:      $V_1 \leftarrow New\_Exp.\text{evaluate}(V_1)$ 
10:     $V_2 \leftarrow New\_Exp.\text{evaluate}(V_2)$ 
11:     $Exp \leftarrow New\_Exp$ 
12:     $Type \leftarrow \text{get\_type}(Exp)$ 
13:   end if
14:    $D \leftarrow D - 1$ 
15: end while

```

and $n'.id = 5$ (i.e., N_5) would evaluate to the same value 1. As a result, this expression is ineffective in ruling out N_5 .

GQS generates complex expressions by recursively nesting templates based on the original property access while ensuring the distinguishability of elements at each nesting level. The pseudocode for this process is provided in Algorithm 2. This algorithm takes as input two sets of graph elements, S_1 and S_2 , one of their common property name P whose value can distinguish these two sets, and the nesting depth D . It outputs an expression Exp that, when instantiated with elements from the respective sets, evaluates to distinct values, denoted as V_1 and V_2 , respectively.

The algorithm begins by initializing the expression Exp with the original property-access expression (line 1). The evaluation results are then initialized by instantiating Exp with elements from the respective sets, S_1 and S_2 (lines 2–3). Next, the algorithm determines Exp 's data type and selects a new expression template that includes a parameter matching this data type (lines 4–6). The new expression is instantiated with the original Exp as its parameter (line 7). If the template requires additional parameters, they are filled randomly. The evaluation results for the new expression are then calculated for both sets (line 8). If the evaluation results overlap, indicating the template cannot differentiate between the elements of S_1 and S_2 , the algorithm selects a new template. Otherwise, Exp is updated with the new expression (line 11), and its data type is updated based on the result of the nested expression (line 12).

This process repeats, with the new Exp serving as a parameter for another expression template until the specified depth D is reached.

Example 3.3. In the aforementioned example, S_1 and S_2 include N_4 and N_5 , respectively, while P represents the property name `id`. GQS first finds an expression template that can accept integers as parameters (i.e., $n'.id$'s type). If `left(m.name, n'.id)` is chosen as the new expression template, GQS checks if it still evaluates to different values when instantiated with $n'.id = 4$ and $n'.id = 5$, respectively. If this holds true, GQS then searches for another

template that can use `left(m.name, n'.id)` as a parameter and remains distinguishable for $n'.id = 4$ and $n'.id = 5$. The process continues until the predefined nesting depth is reached.

4 Implementation

We implement our GQS approach in an automatic testing tool for GDBs that support Cypher queries. The overall codebase consists of 34K lines of Java code.

Integrating Different GDBs. GQS can be easily adapted to test specific GDBs. We have integrated four popular GDBs, namely Neo4j, Memgraph, FalkorDB, and Kùzu, using their Java drivers. Each of the first three GDBs requires around 100 LoC to configure and ensure stable testing. Kùzu requires the database schema information before initializing a random graph, and 150 LoC were needed to meet this requirement. For testing new GDBs, GQS provides interfaces that allow for easy integration of their Java drivers.

Supported Cypher Features. Our implementation covers all 11 clauses and subclauses for data retrieval, as documented in the Cypher Query Language Reference [37]; see also Section 5.3. The only exception is the `MANDATORY MATCH` clause, which is not supported by the four Cypher databases we test (Section 5.1).

All supported data retrieval clauses are synthesized according to the plan described in Section 3.2, except `UNION` and `CALL`. These two clauses are handled separately due to their unique semantics and varied implementations in practice. Specifically, since `UNION` connects two queries, it is added after the synthesis of two separate queries. The `CALL` clause invokes database engine procedures. However, some implementations intentionally deviate from the Language Reference [37]. For example, to retrieve graph element labels, Neo4j and FalkorDB provide the procedure `CALL db.labels()`, which align with the reference, while Kùzu and Memgraph do not provide such functionality. In addition, our graph initializer incorporates all six clauses for writing data to the database. Unlike the data retrieval clauses, these clauses are not used in our ground-truth validation but are instead employed to create and update graphs.

GQS supports an extensive library of 61 functions, as well as aggregation operators, property access operators, and mathematical operators, which are commonly supported by the four tested GDBs. Functions limited to only a few databases are excluded to ensure compatible queries. Subquery and transaction features are currently unsupported. Variable-length patterns are not supported as they could introduce undesired subgraph matches.

Handling GDB-specific Cypher Variations. By default, our implementation synthesizes Cypher queries in line with the Cypher Query Language Reference [37]. However, we also account for variations in how different GDBs implement certain Cypher features. For instance, Kùzu and FalkorDB deviate from the relation uniqueness requirement, allowing the same relation to be matched multiple times within a single query pattern. In the search pattern $(n1)-[e1]-(n2)-[e2]-(n3)$, $e1$ and $e2$ are expected to correspond to different relations in the graph according to the reference. However, in FalkorDB and Kùzu, the same relation can be matched by both $e1$ and $e2$. To address this, we introduce additional `WHERE` predicates to filter out duplicate matches, e.g., `WHERE e1!=e2`.

Table 2: Summary of the tested GDBs.

GDB	GitHub stars	Initial release	Tested version	LoC
Neo4j	13.2K	2007	5.18, 5.20, 5.21.2	1.4M
Memgraph	2.4K	2017	2.13, 2.14.1, 2.15, 2.17	0.2M
Kùzu	1.3K	2022	0.4.2, 0.7.1	11.9M
FalkorDB*	651	2023	4.2.0	2.8M

* RedisGraph, the predecessor of FalkorDB, has received 2K GitHub stars since its initial release in 2018.

Table 3: Summary of the bugs detected by GQS.

GDB	Logic bugs			Other bugs		
	#detected	#confirmed	#fixed	#detected	#confirmed	#fixed
Neo4j	2	2	2	3	3	3
Memgraph	6	6	1	1	1	0
Kùzu	5	5	5	2	2	2
FalkorDB	13	4	0	4	2	1
Total	26	17	8	10	8	6

5 Experiments

In this section, we conduct an extensive assessment of GQS, along with the state-of-the-art logic bug detectors for GDBs. We seek to answer the following questions:

- Q1. Can GQS detect new bugs in production GDBs (Section 5.2)?
- Q2. How do our key design choices contribute to GQS's effectiveness in bug detection (Section 5.3)?
- Q3. Is GQS more effective in finding bugs than the state-of-the-art (Section 5.4)?

5.1 Experimental Setup

We examine recent releases of four popular and heavily tested GDBs: Neo4j [34], the market leader with over two decades of development; Memgraph [31], an in-memory GDB specializing in real-time analytics; FalkorDB [13], the fork of RedisGraph [41], designed for large language models; and Kùzu [24], an emerging embeddable GDB. During our testing, when new versions of the GDBs were released, we deployed new instances to assess them. Table 2 provides details on the tested GDBs. Note that when comparing GQS with the state-of-the-art tools, we also test early releases of Neo4j, Memgraph, and FalkorDB (in this case, RedisGraph), which are not included in Table 2. These releases have been extensively tested using these competing tools; see Section 5.4 for details.

Our experiments were conducted on a Linux workstation running Ubuntu 22.04, with AMD Ryzen 9 7950X and 128GB of memory. We initialize the tested GDBs using random graphs with up to 13 nodes and 500 relations. In addition, we set GQS to perform up to 9 synthesis steps to balance the complexity of test cases with the efficiency of their execution (see Section 5.3). The maximum size of the ground-truth property set is limited to 6. As we will see, this configuration showcases GQS's effectiveness in bug detection and distinguishes it from other testers. To report unique bugs, we manually filter out duplicate test queries that trigger the same bugs.

5.2 Discovering New Bugs

We have detected and reported 36 previously unknown bugs across all four GDBs, as summarized in Table 3. For more details, please refer to our repository. Among these bugs, 26 are logic errors that

yield incorrect query results. Additionally, there are 10 other bugs, including memory corruption, crashes, or unexpected exceptions, which can cause the GDBs to hang indefinitely, consume excessive memory, etc. As of this writing, the developers have confirmed 25 of these issues, with 14 already fixed in the latest releases. In particular, nearly all the bugs discovered in Neo4j, Memgraph, and Kùzu have been confirmed, and the majority have already been fixed in response to our reports. Overall, our results demonstrate GQS's effectiveness in finding various bugs with complex queries.

Interesting Bugs. We presented a representative bug in Section 1. In the following, we provide examples of three other bugs.

Example 5.1. Figure 6 illustrates a Cypher query that triggers a logic bug in Neo4j. This query consists of four main steps, with two **MATCH** clauses searching for the desired patterns and an **UNWIND** clause in between expanding the arrays involved. Instead of returning the correct value of `r4.k191` as specified in the query (line 4), Neo4j incorrectly returns the property value of a node (`n4`) that is not even queried for. Such bugs could result in data leakage or, if exploited, unauthorized data access.

```

1 MATCH (n0 :L11)<-[r0 :T3]-(n1) WHERE (((NOT (NOT ...
2 UNWIND [(r0.k186), 557243387] AS a0
3 MATCH (n2 :L11 :L5)-[r1 :T3]->(n3 :L11), ..., (n7 :L11 :
  L5)-[r4 :T3]->(n8 :L11 :L5 :L4) WHERE ...
4 RETURN (r4.k190) AS a3, (r4.k191) AS a4
5 // expected result: 6 rows of {a3:v6z5e, a4:true} ✓
6 // actual result: 6 rows of {a3:v6z5e, a4:WEJ6Mifdo} ✗

```

Figure 6: A logic bug found in Neo4j: an incorrect return for `r4.k191`'s value. Parts of the query are omitted for simplicity.

Example 5.2. Figure 7 presents a test query that causes Memgraph to incorrectly produce an empty result. The query involves five steps and eight clauses and subclauses, which collectively trigger Memgraph's complex optimization logic. The bug arises from an unexpected optimization combination in the query plan, specifically Cartesian product optimizations combined with filtering.

Fixing this bug proved challenging. Despite quickly initiating an investigation, the developers' progress stalled after one month. As a temporary solution, they recommended disabling Cartesian product optimizations, which compromised performance to ensure correctness. A fix was finally implemented after six months.

```

1 MATCH (n0 :L0 :L6 :L11)<-[r0 :T2]-(n1), (n2 :L6)<-[r1 :T2
  ]-(n3 :L0) WHERE ...
2 UNWIND [-1465465557] AS a0
3 MATCH (n4 :L0)<-[r2 :T2]-(n5 :L0 :L6) WHERE ...
4 UNWIND [(n0.k65)] AS a1
5 RETURN (r1.k86) AS a2, (n3.k4) AS a3, (r1.k87) AS a4
  ORDER BY a4 DESC
6 // expected result: {a2:0spkB, a3:false, a4:SqpUzADY6} ✓
7 // actual result: {} ✗

```

Figure 7: A logic bug in Memgraph caused by unexpected optimization combinations.

While our focus is on logic bugs, GQS can also trigger memory corruption bugs, unexpected exceptions, or GDB crashes. The test oracles for identifying these issues come at no additional cost.

Example 5.3. Figure 8 shows a query that exposes a memory leak in the latest release of Memgraph. This issue originates from the

replace function, which is currently underspecified for handling empty strings. Consequently, when the query attempts to match an empty string in the string “tS15G” and replaces it with another string “U1lsWFvRw”, Memgraph hangs indefinitely and consumes over 50GB of memory within five minutes.

This bug is severe. It can significantly affect GDB availability and lead to security vulnerabilities like denial-of-service attacks. Notably, it had persisted for over three years since Memgraph’s earliest publicly accessible release, before being discovered by GQS.

```
1 WITH replace('tS15G', '', 'U1lsWFvRw') AS a0
2 RETURN a0
3 // expected result: {a0:'tS15G'} (empty string ignored)
4 // or a parse error (empty string disallowed) ✓
5 // actual result: Memgraph hangs, consuming memory ✘
```

Figure 8: A memory leak uncovered in Memgraph due to underspecified handling of empty strings.

5.3 A Closer Look at GQS

We have demonstrated GQS’s effectiveness in bug detection. To further assess the contributions of our key design choices to the bugs found, we analyze all 36 bug-triggering test queries with respect to five aspects: synthesis steps, Cypher features, data dependencies, search patterns, and nested expressions.

Overall, the majority of the detected bugs involve many steps, diverse search patterns, deeply nested expressions, rich clause types, and complex cross-clause references within the test queries. This underscores the importance of complex queries in effectively triggering bugs, which is the focus of our GQS approach.

Synthesis Steps. GQS generates queries in a stepwise manner. As shown in Figure 9, 80% of the bugs found are triggered by queries synthesized with at least three steps, with those composed of four to six steps being particularly effective. This result is consistent with the expectation that more complex queries are more likely to cover corner cases in, e.g., pattern searching (Example 5.1) and combining optimizations (Example 5.2). In contrast, existing metamorphic testers often use only one or two clauses, potentially missing bugs.

However, test queries with fewer steps still account for a non-negligible number of bugs. These queries are often sufficient to expose errors in functions or operators, as illustrated in Example 5.3.

More steps necessitate additional synthesis and execution time, as confirmed by our analysis presented in Figure 9. Specifically, as the number of steps increases, the number of test queries completed per second decreases. For example, queries synthesized with nine steps require 6.6 times more execution time on average compared to those synthesized with only three steps. Excessively increasing the steps reduces the bug-finding performance. In practice, users of GQS can configure the number of synthesis steps to strike a balance between effectiveness and efficiency in bug detection.

Cypher Features. GQS-synthesized queries encompass a wide range of Cypher features, including various clauses. As indicated by our analysis, all of these clauses are involved in the bug-triggering test queries. Note that our analysis excludes data manipulation clauses like **CREATE** and **SET** as they are not relevant to data querying (or ground truth validation).

A clause can occur multiple times per query due to the random assignment at each synthesis step. As shown in Figure 10, the **MATCH**

clause appears frequently, with over 70 aggregated occurrences across all the queries. This is expected because pattern matching underlies GDB querying. Note that the **WHERE** clause appears more than 100 times as it serves as the filtering subclause for both **MATCH** and **WITH**. Figure 11 presents the number of bugs related to different clauses. Most of the bugs involve the canonical Cypher skeleton **MATCH-WHERE-RETURN**; however, the number of bugs associated with other clauses is also significant. For example, 24 out of 36 bugs are triggered by queries containing the **ORDER BY** or **WITH** clauses.

In addition, a substantial number of Cypher functions, 32 in total, are involved in the reported bugs. See our repository for details.

Cross-Step Dependencies. During the synthesis of test queries, GQS establishes complex data dependencies across multiple steps. Figure 12 shows that, although a significant number of bugs involve fewer than 20 dependencies, over 61% of bugs are triggered by test queries with more than 20 dependencies. These dependency-heavy queries, such as the one shown in Figure 1, often leads to problematic execution paths in query planners during optimization, resulting in incorrect query outcomes.

Search Patterns and Nested Expressions. We also analyze the variety of search patterns used and the depth of nested expressions involved in the bug-triggering test queries. Figure 13 presents the distribution of bugs based on the number of patterns used. Notably, two-thirds of the bugs are triggered by test queries containing more than three distinct patterns.

Nested expressions also play a crucial role in triggering bugs. As shown in Figure 14, 83% of the bugs are associated with queries that contain more than five levels of nested expressions.

5.4 Comparison with State-of-the-Art

We compare GQS with five state-of-the-art logic bug detectors.

- The metamorphic bug detectors, GDBMeter [22], Gamera [57], GQT [19], and GRev [29], each applying specifically designed query rewrite rules to identify discrepancies between the result of the original query and that of the rewritten query.
- The differential tester GDsmith [17], which detects logic bugs by comparing the results of the same query across multiple GDBs.

5.4.1 Bug Latency. While GDBs such as Neo4j, Memgraph, and RedisGraph (FalkorDB’s predecessor) have been extensively evaluated by the state-of-the-art testers, all of the bugs we report are previously unknown. In line with common practice [21, 29], we analyze the latencies of the bugs found by GQS to determine how many have been overlooked by prior testing efforts. By latency, we refer to the time span between the earliest version of the GDB in which the bug was introduced and when the bug was first discovered by us. If this version predates the versions tested by the competing tools, it indicates that our tool has uncovered long-latent bugs that were missed by these tools. Our analysis excludes Kuzu as it is a relatively new GDB that has not yet been tested by existing tools. This results in a total of 29 bugs across the remaining three GDBs.

As shown in Table 4, over half of the 29 bugs reported by our tool were introduced in versions prior to those tested by the state-of-the-art. In particular, nearly all of the 17 bugs found in FalkorDB (see Table 2) were already present in RedisGraph versions predating those assessed by these tools. The average latency for these missed bugs ranges between two and four years, with the longest-latent

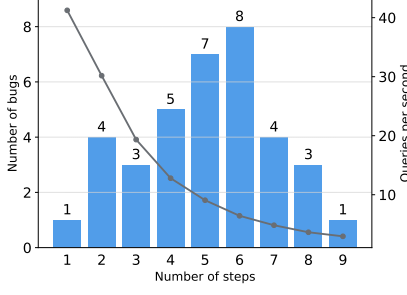


Figure 9: Bug detection effectiveness (bars) and efficiency (line) as a function of the number of synthesis steps.

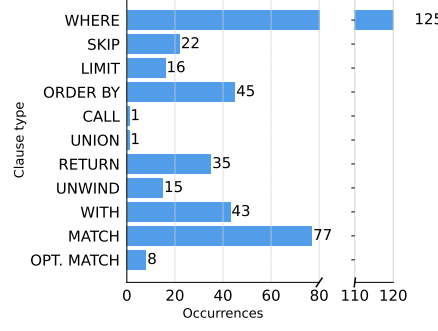


Figure 10: The aggregated number of clauses involved in the bug-triggering test queries.

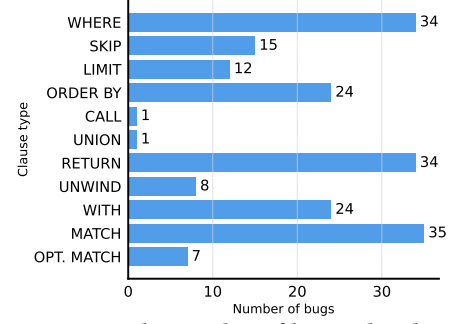


Figure 11: The number of bugs related to different types of clauses.

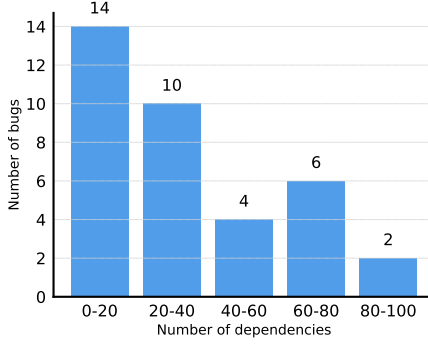


Figure 12: Distribution of bugs based on the number of dependencies involved.

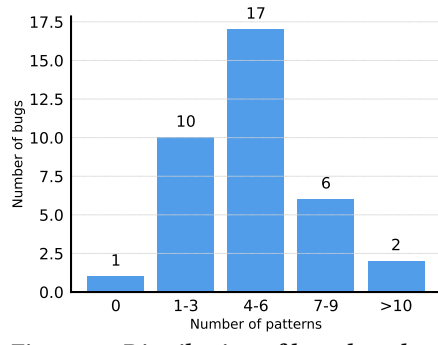


Figure 13: Distribution of bugs based on the number of patterns involved.

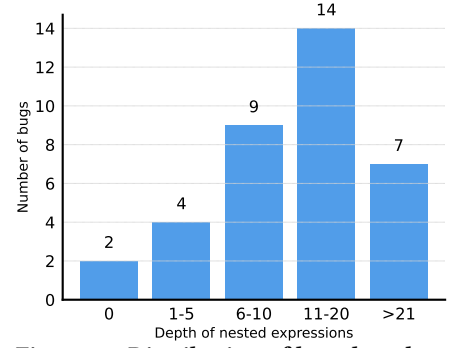


Figure 14: Distribution of bugs based on the depth of nested expressions.

bug having been introduced five years ago. Overall, this underscores our tool's strength in uncovering deeply hidden bugs.

Table 4: Bugs missed by existing testers and their latencies. GDBMeter, Gamera, and GQT did not support Memgraph.

Tester	Neo4j	Memgraph	RedisGraph*	Total
GDsmith	1	2	15	18
GDBMeter	1	–	15	16
Gamera	2	–	16	18
GQT	1	–	16	17
GRev	2	2	16	20
Avg. latency (yrs)	2.2	3.4	4.0	–
Max. latency (yrs)	2.7	3.4	5.0	–

* All tools tested earlier versions of FalkorDB, i.e., RedisGraph.

In the remainder of this section, we explore why our tool is more effective at detecting bugs compared to existing testers.

5.4.2 Test Query Complexity. Generating complex queries is essential for extensively-testing GDBs and triggering logic bugs, as we have already seen in Section 5.3. Therefore, we compare the queries generated by GQS and existing tools with respect to four aspects: patterns, expressions, clauses, and dependencies.

We randomly select 10,000 test queries generated by each tool and use the Cypher query parser [25] to convert them into abstract syntax trees [4]. We then iterate through these trees to collect the following metrics per query: (i) the number of patterns involved, (ii) the maximum depth of nested expressions, (iii) the number of

Table 5: Comparison on test query complexity.

Tester	Pattern	Expression	Clause	Dependency
GDsmith	4.96	3.68	6.39	21.75
GDBMeter	0.86	2.24	1.94	1.97
Gamera	0.83	1.39	1.92	1.89
GQT	1.03	2.87	3.39	3.43
GRev	6.69	5.26	6.49	28.41
GQS	8.14	7.82	6.50	56.02

clauses involved, and (iv) the number of cross-clause data references. We report the average for each metric, as shown in Table 5.

Overall, the test queries synthesized by our tool GQS exhibit substantially greater complexity than those generated by existing tools. In particular, each GQS-generated query utilizes an average of 8.14 patterns, which is approximately 180% more than the average of the other tools. GQS's queries also reach a maximum depth of 7.82 for nested expressions, around 50% deeper than the second-best tool GRev. Moreover, GQS demonstrates significantly higher complexity in data dependencies, with around 100% more cross-clause references compared to GRev. Note that the queries generated by GDBMeter, Gamera, and GQT are significantly less complex. This is partially due to the constraints imposed by their metamorphic oracles, which require test generation to adhere to predefined rewrite rules.

5.4.3 Test Oracle Effectiveness. While GQS is capable of generating more complex queries than existing tools, for a fair comparison

of test oracles, we integrate our bug-triggering test queries with their respective oracles. Our evaluation focuses solely on whether these tools can identify the bugs when provided with the same test queries. Specifically, for metamorphic testers, we apply their rewriting rules to these queries and check whether the transformed queries reveal violations of metamorphic relations. For the differential tester GDsmith, we execute the queries across different GDBs and check for any discrepancies.

Missed Bugs. Overall, metamorphic oracles are insensitive to certain types of bugs, especially when their root causes are unrelated to the predefined metamorphic relations. When comparing with GDBMeter and GRev, we are able to quantify the number of missed bugs because GDBMeter’s metamorphic relations are straightforward to implement, and GRev provides direct interfaces for applying its rewrite rules to given queries. Out of 26 logic bugs detected by QGS, GDBMeter and GRev could only identify 11 and 3, respectively, even when using the bug-triggering test cases.

Example 5.4. Figure 15 shows an example bug found by QGS that is missed by GDBMeter. When running Memgraph with the QGS-synthesized test query (lines 2–3), it produces an incorrect empty output, instead of the expected result (line 14). Upon applying GDBMeter’s ternary logic partitioning [22] to the test query, all three partitioned queries (lines 5–13) also result in empty results. The union of these results matches the incorrect output (namely GDBMeter’s oracle), causing GDBMeter to mistakenly pass the test. The key factor triggering this bug lies in the **WITH** clause, which is not accounted for by GDBMeter’s metamorphic rules.

```

1 // CySy-synthesized test query
2 MATCH (n0)-[r0]->(n1) WITH r0 ...
3 WHERE ("1"<n0.k99) RETURN r0.id AS a0
4
5 // GDBMeter-rewritten query #1 (NOT)
6 MATCH (n0)-[r0]->(n1) WITH r0 ...
7 WHERE NOT ("1"<n0.k99) RETURN r0.id AS a0
8 // GDBMeter-rewritten query #2 (IS NULL)
9 MATCH (n0)-[r0]->(n1) WITH r0 ...
10 WHERE ("1"<n0.k99) IS NULL RETURN r0.id AS a0
11 // GDBMeter-rewritten query #3 (TRUE)
12 MATCH (n0)-[r0]->(n1) WITH r0 ...
13 WHERE TRUE RETURN r0.id AS a0
14 // expected result: {a0:20} ✓
15 // actual result (serving as GDBMeter's oracle): {} ✗
16 // GDBMeter: {}U{}U{}={ } (matching the incorrect oracle)

```

Figure 15: A Memgraph bug found by QGS that cannot be detected by applying GDBMeter’s metamorphic rules.

Unlike the above two metamorphic testers, Gamera and GQT do not provide interfaces for directly applying their metamorphic relations to specific queries. In addition, their query rewrites incorporate a degree of randomness. For instance, GQT randomly adds labels to nodes during query rewrites, resulting in infinitely many transformations. Consequently, it is infeasible to precisely quantify the number of bugs they might overlook. Therefore, we manually analyze their predefined metamorphic relations and observe that these tools are likely to miss some bugs reported by our tool, as their root causes are unrelated to the relations.

Example 5.5. Figure 16 illustrates a representative bug found in the latest version of FalkorDB, which is unlikely to be detected by

Gamera or GQT. This bug’s root cause lies in FalkorDB’s mishandling of **UNWIND**’s semantics, resulting in some records not being fetched. Gamera’s and GQT’s oracles are unlikely to identify this bug because their metamorphic rules do not cover **UNWIND**.

```

1 UNWIND [1,2,3] AS a0
2 MATCH (n2 :L12)-[r1]-(n3) WHERE (((r1.id) = 13) AND ...
3 RETURN a0
4 // expected result: [{a0:1},{a0:2},{a0:3}] ✓
5 // actual result: [{a0:1}] ✗

```

Figure 16: A bug found by QGS in FalkorDB’s latest version that is unlikely to be detected by Gamera or GQT.

False Alarms. For the differential tester GDsmith, feeding it with QGS-generated bug-triggering queries resulted in no missed bugs. This is expected since our tested GDBs do not share a substantial common codebase; therefore, the same bugs do not exist across these GDBs. However, we observe a significant number of false positives, which may limit GDsmith’s practical applicability. In particular, running GDsmith for 24 hours on the latest versions of Neo4j and Memgraph resulted in 1192 bug reports, of which 1160 are false positives, yielding an approximate 98% false positive rate. Such high false alarm rates mainly come from exceptions raised by invalid queries and output format differences.

24-Hour Empirical Testing. What about the bug detection effectiveness of each tool on its own? We conduct a 24-hour empirical analysis of all testers using their respective query generators.

Our result, presented in Table 6, demonstrates QGS’s superior effectiveness compared to the other tools. Overall, QGS identifies a total of 13 bugs, including 11 logic bugs. Even the second-best tool, GDsmith, finds approximately 50% fewer bugs. Note that Table 3 (Section 5.2) reports a greater number of bugs than mentioned here, as our overall testing process spanned several months.

Table 6: Bugs detected over a 24-hour testing.

Tester	Neo4j	Memgraph	FalkorDB	Total
GDsmith	1 (1)	2 (1)	4 (4)	7 (6)
GDBMeter	1 (0)	–	1 (0)	2 (0)
Gamera	0 (0)	–	1 (0)	1 (0)
GQT	0 (0)	–	5 (5)	5 (5)
GRev	0 (0)	1 (1)	1 (1)	2 (2)
QGS	2 (1)	3 (2)	8 (8)	13 (11)

X (Y): among the X bugs found in total, Y bugs are logic bugs.

6 Related Work

Graph Database Testing. We have already discussed the state-of-the-art tools for finding GDB logic bugs in Section 1. Overall, these tools employ either differential testing, as in GDsmith [17] and Grand [17], or metamorphic testing, as in Gamera [57], GDBMeter [22], GRev [29], and GQT [19]; however, neither approach can establish the ground truth for their test oracles. As a result, GDB logic bugs may be overlooked or incorrectly reported, as demonstrated by our experiments (in Section 5.4). Our evaluation shows that our QGS approach is more effective in both detecting logic bugs and avoiding false alarms.

Moreover, compared to existing testers, GQS is capable of generating more complex test queries, as demonstrated in Section 5.3 and Table 5. These queries play a crucial role in detecting incorrect GDB behaviors as well as crashes and exceptions.

Relational Database Testing. In recent years, there has been a plethora of logic bug detectors [3, 16, 20, 21, 43, 44, 47, 48, 51] for relational databases (RDBs). While GDBs and RDBs differ fundamentally in many aspects, their bug-detection techniques are closely related. For instance, GDBMeter adapts the metamorphic idea of TLP [43] designed for testing RDBs, which partitions a query based on three-valued logic. Similar to GDsmith, DQE [47] also leverages differential testing, but specifically for examining predicate operations in RDBs. In fact, much like in GDB testing, most of these RDB testers adopt either differential or metamorphic oracles. Therefore, they inherently lack the ground truth for test queries.

Akin to our GQS approach, PQS [44], Pinolo [16], and TQS [51] test RDBs by synthesizing SQL queries. However, PQS and Pinolo cannot establish the ground truth. Both methods rely on subset relationships as test oracles; satisfying these relationships does *not* guarantee the correctness of individual queries. TQS targets logic bugs in join optimizations. It synthesizes join queries from multiple tables. These tables are derived from a single given table based on normal forms *specific to* RDBs, which serves as the ground truth.

Database Query Synthesis. SQL or datalog query synthesis has been widely studied over the decades, finding applications in areas such as database education, query optimization, and data migration. However, existing synthesizers [32, 33, 39, 46, 49, 52, 53, 55] are *not* suitable for solving our problem. First, SQL or datalog queries are fundamentally different from graph queries like Cypher. Moreover, the application domains differ. For example, simple queries are more explainable for educational purposes [53]; whereas our goal is to synthesize complex queries that can effectively test GDBs.

7 Discussion and Concluding Remark

We have presented (i) GQS, the first automatic testing approach for detecting logic bugs in GDBs with respect to an established ground truth, and (ii) an instantiation of GQS, incorporating the first Cypher query synthesizer tailored for GDBs. We have utilized GQS to uncover many previously unknown, long-latent bugs in production GDBs and demonstrated its superior effectiveness in bug detection compared to the state-of-the-art. Our approach is conceptually simple, facilitating its adoption in practice, yet highly effective, contributing to more reliable GDBs.

In the following, we discuss limitations and future work.

Limitations. While testing can find bugs, it can generally not guarantee their absence. Our GQS approach, like other bug detectors, shares this inherent limitation. However, as demonstrated by our analysis, GQS is more effective than them in identifying logic bugs. Due to the limitations of black-box testing, GQS may generate duplicate bug reports, as bugs with a common root cause can be triggered by seemingly distinct test cases. Currently, we rely on manual analysis to deduplicate these reports. Automatic bug deduplication remains an open challenge [3, 11], and we consider addressing this as part of future work.

In addition, GQS relies on a correct implementation that accurately reports logic bugs. If the implementation is flawed, false

alarms might be produced. We have not observed such issues, and we argue it rarely happens as the query synthesis is straightforward based on the plan. For instance, to synthesize a **MATCH** clause according to the plan, i.e., introduce a set of given graph elements, directly encoding the subgraphs containing these elements in the search pattern format will suffice. Compared with a full-reference implementation, which must traverse the entire graph according to that pattern, this plan-guided synthesis approach significantly simplifies the implementation's complexity. In case of false alarms, GQS provides auxiliary logs to identify the problematic features.

GQS focuses on the common Cypher features shared across the tested GDBs to ensure compatible test queries, and it does not yet accommodate features specific to some GDBs, e.g., label expressions unique to Neo4j. Currently, GQS does not support subqueries or OLAP (Online Analytical Processing) queries. We plan to extend GQS with these additional features in future work.

Beyond Logic Bugs. Graph query generation is fundamental to GDB testing, not only for detecting logic bugs but also for assessing other properties. As already demonstrated by our experiments, the complex queries we synthesized are effective in triggering GDB crashes or unexpected exceptions (10 in total across four GDBs). Additionally, our query synthesizer can serve as a basis for examining performance issues in GDBs, such as inefficient query processing or optimization. The challenge lies in the current lack of ground truth on the expected execution time, where the cardinality estimation approach [42] designed for RDBs may provide an interesting direction for future research.

Recent years have also seen advances in black-box checking of isolation guarantees [8, 15, 18, 23, 26, 50, 54], such as *serializability* [38], *snapshot isolation* [6], and weaker levels [5, 27]. These efforts primarily focus on RDBs and their concurrency control mechanisms for transactions (assuming the correctness for individual queries). Our work can be naturally extended to address this problem for GDBs, e.g., checking serializability claimed by Neo4j or snapshot isolation claimed by Memgraph. This can be done by wrapping generated Cypher queries as transactions and utilizing existing oracles like Adya's theory [2] or Biswas and Enea's axioms [8] to validate the fulfillment of different isolation guarantees.

Beyond Cypher. While Cypher is the *de facto* graph query language, other languages, such as Gremlin [45], are also widely used. To extensively test Gremlin-based GDBs, one can leverage the advanced *Cypher for Gremlin* compiler [36] developed by openCypher to translate our synthesized Cypher queries into Gremlin queries. Alternatively, our methodology can be applied to design a dedicated Gremlin synthesizer that supports its full range of features [14]. This is feasible because Gremlin also employs a chain structure in its queries, comprising steps for individual graph traversal operations.

The GQL standard [1] has recently been published, marking a significant milestone in the database world. On one hand, our synthesizer will continue to be in action, as, e.g., Neo4j's compliance with GQL will not prevent any existing Cypher queries from functioning [28]. On the other hand, our approach and synthesizer lay the groundwork for effectively testing GQL-based databases in the future, as both languages naturally and deliberately converge, sharing a largely identical core syntax [28, 40].

References

- [1] ISO/IEC 39075:2024. Accessed in July, 2024. Information technology – Database languages – GQL. <https://www.iso.org/standard/76120.html>.
- [2] Atul Adya. 1999. *Weak consistency: a generalized theory and optimistic implementations for distributed transactions*. Ph. D. Dissertation. Massachusetts Institute of Technology, Department of Electrical Engineering and Computer Science.
- [3] Jinsheng Ba and Manuel Rigger. 2024. Keep It Simple: Testing Databases via Differential Query Plans. *Proc. ACM Manag. Data* 2, 3, Article 188 (2024).
- [4] John W. Backus. 1959. The syntax and semantics of the proposed international algebraic language of the Zurich ACM-GAMM Conference. In *Information Processing, Proceedings of the 1st International Conference on Information Processing, UNESCO, Paris 15-20 June 1959*. UNESCO (Paris), 125–131.
- [5] Peter Bailis, Aaron Davidson, Alan D. Fekete, Ali Ghodsi, Joseph M. Hellerstein, and Ion Stoica. 2013. Highly Available Transactions: Virtues and Limitations. *Proc. VLDB Endow.* 7, 3 (2013), 181–192.
- [6] Hal Berenson, Phil Bernstein, Jim Gray, Jim Melton, Elizabeth O’Neil, and Patrick O’Neil. 1995. A critique of ANSI SQL isolation levels. *ACM SIGMOD Record* 24, 2 (1995), 1–10.
- [7] Maciej Besta, Robert Gerstenberger, Emanuel Peter, Marc Fischer, Michał Podstawski, Claude Barthels, Gustavo Alonso, and Torsten Hoefer. 2023. Demystifying Graph Databases: Analysis and Taxonomy of Data Organization, System Designs, and Graph Queries. *ACM Comput. Surv.* 56, 2, Article 31 (sep 2023).
- [8] Ranadeep Biswas and Constantin Enea. 2019. On the complexity of checking transactional consistency. *Proc. ACM Program. Lang.* 3, OOPSLA (2019), 165:1–165:28.
- [9] Angela Bonifati, George H. L. Fletcher, Hannes Voigt, and Nikolay Yakovets. 2018. *Querying Graphs*. Morgan & Claypool Publishers.
- [10] Tsong Yueh Chen, S. C. Cheung, and Siu-Ming Yiu. 2020. Metamorphic Testing: A New Approach for Generating Next Test Cases. *CoRR* abs/2002.12543 (2020). [arXiv:2002.12543](https://arxiv.org/abs/2002.12543) <https://arxiv.org/abs/2002.12543>
- [11] Yang Chen, Alex Groce, Chaoyang Zhang, Weng-Keen Wong, Xiaoli Z. Fern, Eric Eide, and John Regehr. 2013. Taming compiler fuzzers. In *PLDI ’13*, Hans-Juergen Boehm and Cormac Flanagan (Eds.). ACM, 197–208.
- [12] Cypher. Accessed in July, 2024. <https://neo4j.com/docs/cypher-manual/current/introduction/>.
- [13] FalkorDB. Accessed in July, 2024. <https://www.falkordb.com/>.
- [14] Cypher for Gremlin. Accessed in July, 2024. Cucumber Report. <https://opencypher.github.io/cypher-for-gremlin/test-reports/1.0.4/cucumber-html-reports/overview-features.html>.
- [15] Long Gu, Si Liu, Tiancheng Xing, Hengfeng Wei, Yuxing Chen, and David Basin. 2024. IsoVista: Black-box Checking Database Isolation Guarantees. *Proc. VLDB Endow.* 17, 12 (2024).
- [16] Zongyin Hao, Quanfeng Huang, Chengpeng Wang, Jianfeng Wang, Yushan Zhang, Rongxin Wu, and Charles Zhang. 2023. Pinolo: Detecting Logical Bugs in Database Management Systems with Approximate Query Synthesis. In *USENIX ATC ’23*. USENIX Association, 345–358.
- [17] Ziyue Hua, Wei Lin, Luyao Ren, Zongyang Li, Lu Zhang, Wenpin Jiao, and Tao Xie. 2023. GDsmith: Detecting Bugs in Cypher Graph Database Engines. In *ISSTA ’23*. ACM, 163–174.
- [18] Kaile Huang, Si Liu, Zheng Chen, Hengfeng Wei, David A. Basin, Haixiang Li, and Anqun Pan. 2023. Efficient Black-box Checking of Snapshot Isolation in Databases. *Proc. VLDB Endow.* 16, 6 (2023), 1264–1276.
- [19] Yuancheng Jiang, Jiahao Liu, Jinsheng Ba, Roland H. C. Yap, Zhenkai Liang, and Manuel Rigger. 2024. Detecting Logic Bugs in Graph Database Management Systems via Injective and Surjective Graph Query Transformation. In *ICSE ’24*. ACM, Article 46.
- [20] Zu-Ming Jiang, Si Liu, Manuel Rigger, and Zhendong Su. 2023. Detecting Transactional Bugs in Database Engines via Graph-Based Oracle Construction. In *OSDI ’23*. 397–417.
- [21] Zu-Ming Jiang and Zhendong Su. 2024. Detecting Logic Bugs in Database Engines via Equivalent Expression Transformation. In *OSDI ’24*. USENIX Association, 821–835.
- [22] Matteo Kamm, Manuel Rigger, Chengyu Zhang, and Zhendong Su. 2023. Testing Graph Database Engines via Query Partitioning. In *ISSTA ’23*. ACM, 140–149.
- [23] Kyle Kingsbury and Peter Alvaro. 2020. Elle: Inferring Isolation Anomalies from Experimental Observations. *Proc. VLDB Endow.* 14, 3 (2020), 268–280.
- [24] Kuzu. Accessed in July, 2024. <https://kuzudb.com/>.
- [25] Cypher Parser Library. Accessed in July, 2024. [libcypher-parser](https://github.com/cleishm/libcypher-parser) <https://github.com/cleishm/libcypher-parser>
- [26] Si Liu, Long Gu, Hengfeng Wei, and David Basin. 2024. Plume: Efficient and Complete Black-box Checking of Weak Isolation Levels. *Proc. ACM Program. Lang.* 8, OOPSLA2 (2024).
- [27] Si Liu, Luca Multazzu, Hengfeng Wei, and David A. Basin. 2024. NOC-NOC: Towards Performance-optimal Distributed Transactions. *Proc. ACM Manag. Data* 2, 1, Article 9 (mar 2024).
- [28] Valerio Malenchino. Accessed in July, 2024. GQL is Here: Your Cypher Queries in a GQL World. <https://neo4j.com/blog/cypher-gql-world/>.
- [29] Qiuyang Mang, Aoyang Fang, Boxi Yu, Hanfei Chen, and Pinjia He. 2024. Testing Graph Database Systems via Equivalent Query Rewriting. In *ICSE ’24*. ACM, Article 143, 12 pages.
- [30] William M. McKeeman. 1998. Differential Testing for Software. *Digit. Tech. J.* 10, 1 (1998), 100–107.
- [31] Memgraph. Accessed in July, 2024. <https://memgraph.com/>.
- [32] Jonathan Mendelson, Aaditya Naik, Mukund Raghothaman, and Mayur Naik. 2021. GENSYNTH: Synthesizing Datalog Programs without Language Bias. *Proceedings of the AAAI Conference on Artificial Intelligence* 35, 7 (May 2021), 6444–6453.
- [33] Aaditya Naik, Aalok Thakkar, Adam Stein, Rajeev Alur, and Mayur Naik. 2023. Relational Query Synthesis $\bowtie$ Decision Tree Learning. *Proc. VLDB Endow.* 17, 2 (2023), 250–263.
- [34] Neo4j. Accessed in July, 2024. <https://neo4j.com/>.
- [35] openCypher. Accessed in July, 2024. <https://opencypher.org/>.
- [36] openCypher. Accessed in July, 2024. Cypher for Gremlin. <https://github.com/opencypher/cypher-for-gremlin>.
- [37] openCypher. Accessed in July, 2024. Cypher Query Language Reference, Version 9. <https://s3.amazonaws.com/artifacts.opencypher.org/openCypher9.pdf>.
- [38] Christos H Papadimitriou. 1979. The serializability of concurrent database updates. *Journal of the ACM (JACM)* 26, 4 (1979), 631–653.
- [39] Mukund Raghothaman, Jonathan Mendelson, David Zhao, Mayur Naik, and Bernhard Scholz. 2019. Provenance-guided synthesis of Datalog programs. *Proc. ACM Program. Lang.* 4, POPL, Article 62 (dec 2019), 27 pages.
- [40] Philip Rathle and Brad Bebee. Accessed in July, 2024. GQL: The ISO standard for graphs has arrived. <https://aws.amazon.com/blogs/database/gql-the-iso-standard-for-graphs-has-arrived/>.
- [41] RedisGraph. Accessed in July, 2024. <https://github.com/RedisGraph/RedisGraph>.
- [42] Manuel Rigger and Zhendong Su. 2020. Detecting optimization bugs in database engines via non-optimizing reference engine construction. In *ESEC/FSE ’20*. ACM, 1140–1152.
- [43] Manuel Rigger and Zhendong Su. 2020. Finding bugs in database systems via query partitioning. *Proc. ACM Program. Lang.* 4, OOPSLA, Article 211 (nov 2020), 30 pages.
- [44] Manuel Rigger and Zhendong Su. 2020. Testing Database Engines via Pivoted Query Synthesis. In *OSDI ’20*. USENIX Association, 667–682.
- [45] Marko A. Rodriguez. 2015. The Gremlin graph traversal machine and language (invited talk). In *Proceedings of the 15th Symposium on Database Programming Languages (DBPL 2015)*. ACM, 1–10.
- [46] Xujie Si, Woosuk Lee, Richard Zhang, Aws Albarghouthi, Paraschos Koutris, and Mayur Naik. 2018. Syntax-guided synthesis of Datalog programs. In *FSE 2018*. ACM, 515–527.
- [47] Jiansen Song, Wensheng Dou, Ziyu Cui, Qianwang Dai, Wei Wang, Jun Wei, Hua Zhong, and Tao Huang. 2023. Testing Database Systems via Differential Query Execution. In *ICSE ’23*. IEEE Press, 2072–2084.
- [48] Jiansen Song, Wensheng Dou, Yu Gao, Ziyu Cui, Yingying Zheng, Dong Wang, Wei Wang, Jun Wei, and Tao Huang. 2024. Detecting Metadata-Related Logic Bugs in Database Systems via Raw Database Construction. *Proc. VLDB Endow.* 17, 8 (may 2024), 1884–1897.
- [49] Keita Takenouchi, Takashi Ishio, Joji Okada, and Yuji Sakata. 2021. PATSQL: Efficient Synthesis of SQL Queries from Example Tables with Quick Inference of Projected Columns. *Proc. VLDB Endow.* 14, 11 (2021), 1937–1949.
- [50] Cheng Tan, Changgeng Zhao, Shuai Mu, and Michael Walfish. 2020. COBRA: Making Transactional Key-Value Stores Verifiably Serializable (OSDI ’20). USENIX Association, Article 4.
- [51] Xiu Tang, Sai Wu, Dongxiang Zhang, Feifei Li, and Gang Chen. 2023. Detecting Logic Bugs of Join Optimizations in DBMS. *Proc. ACM Manag. Data* 1, 1, Article 55 (may 2023), 26 pages.
- [52] Aalok Thakkar, Aaditya Naik, Nathaniel Sands, Rajeev Alur, Mayur Naik, and Mukund Raghothaman. 2021. Example-guided synthesis of relational queries. In *PLDI ’21*. ACM, 1110–1125.
- [53] Chenglong Wang, Alvin Cheung, and Rastislav Bodik. 2017. Synthesizing highly expressive SQL queries from input-output examples. In *PLDI ’17*. ACM, 452–466.
- [54] Jian Zhang, Ye Ji, Shuai Mu, and Cheng Tan. 2023. Viper: A Fast Snapshot Isolation Checker. In *EuroSys ’23*. ACM, 654–671.
- [55] Sai Zhang and Yuyin Sun. 2013. Automatically synthesizing SQL queries from input-output examples. In *ASE ’13*. IEEE, 224–234.
- [56] Yingying Zheng, Wensheng Dou, Yicheng Wang, Zheng Qin, Lei Tang, Yu Gao, Dong Wang, Wei Wang, and Jun Wei. 2022. Finding bugs in Gremlin-based graph database systems via randomized differential testing. In *ISSTA ’22*. ACM, 302–313.
- [57] Zeyang Zhuang, Penghui Li, Pingchuan Ma, Wei Meng, and Shuai Wang. 2024. Testing Graph Database Systems via Graph-Aware Metamorphic Relations. *Proc. VLDB Endow.* 17, 4 (mar 2024), 836–848.